

Arbeitsexposé

KI-gestützte 3D-Rekonstruktion linearer Infrastruktur: Evaluierung einer Docker-basierten Scan-to-BIM Pipeline mittels SAM 3 und Gaussian Splatting

Bachelor- und Projektarbeit

19. Mai 2026

1 Einleitung und Forschungsfrage

Die präzise Erfassung schwer zu modellierender, extrem dünner Objekte (wie Stromerkabel in Baugräben) stellt für klassische photogrammetrische Verfahren (SfM) eine große Herausforderung dar. Ziel dieser Arbeit ist die Entwicklung und Evaluierung eines SScan-to-BIMWorkflows, der die Vorzüge der 2D-KI-Segmentierung (Segment Anything Model 3) mit modernsten 3D-Rekonstruktionsmethoden (3D Gaussian Splatting via SuGaR) verbindet.

Wie aktuelle Forschungen belegen, erzeugen 3D Gaussian Splatting Algorithmen durch mathematische Wahrscheinlichkeitsfunktionen hochgradig detailreiche, fotorealistische Visualisierungen bei geringer Rechenlast. Sie werden zunehmend im Ingenieurwesen und zur Generierung von Orthofotos (z.B. Integration in ArcGIS Pro) eingesetzt. Jedoch fehlt 3DGS-Modellen die inhärente Georeferenzierung. Die zentrale Forschungsfrage lautet daher: *Lässt sich die 3D-Rekonstruktion linearer Infrastruktur durch eine vorgelagerte 2D-Segmentierung und modifizierte 3DGS-Trainingsfunktionen (MMask-Loss Hack) so optimieren und georeferenzieren, dass strenge ingenieurstechnische Toleranzen eingehalten werden?*

2 Praxisbezug und Erfolgskriterien

Die Pipeline wird direkt für den Anwendungsfall der TenneT (Übertragungsnetzbetreiber) konzipiert.

- **Geometrische Toleranz:** Eine Abweichung von maximal ± 10 cm in Lage und Höhe gegenüber der realen Kabeltrasse.
- **Georeferenzierung & GIS-Integration:** Das finale 3D-Modell muss präzise in einem UTM-Koordinatensystem vorliegen, um den direkten Import in Geoinformationssysteme (z.B. ArcGIS Pro) zu gewährleisten.

- **Wirtschaftliche Evaluierung:** Eine Kostengegenüberstellung des Drohnen- und Serververrechnungsaufwands im Vergleich zur klassischen, terrestrischen Vermessung.

3 Methodik und Architektur (Workflow v3)

Der Ansatz basiert auf einer klaren Trennung von 2D-Bildverarbeitung und 3D-Meshing.

3.1 Software-Repositories

- **SAM 3 (Meta):** Zuständig für Concept-Prompting und Video-Tracking.
- **COLMAP:** Generierung der Kameraposen auf Basis der *unmaskierten* Originalbilder. Die Pipeline nutzt ausschließlich das schnelle *Structure from Motion* (SfM) von COLMAP. Die rechenintensive dichte Rekonstruktion (MVS) wird vollständig übersprungen.
- **Segment-then-Splat (STS):** Übernimmt das objektspezifische 3DGS-Training (Lu et al., NeurIPS 2025). STS teilt die initialen COLMAP-Gaussians anhand der SAM 3 Masken in objektspezifische Sets auf und trainiert diese mit einem dedizierten Object-specific Loss ($\mathcal{L}_{obj}$), ohne die Ground-Truth-Bilder zu manipulieren. Das Ergebnis ist ein vollständiges 3DGS-Modell, in dem jeder Gaussian eine Object-ID trägt. SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2.
- **SuGaR:** Surface extraction with Gaussian Splatting. Übernimmt den vortrainierten STS-Checkpoint und wendet geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. SuGaR zwingt die von STS bereits sauber zugewiesenen Gaussians dazu, sich flach an die reale 3D-Oberfläche anzuschmiegen, und extrahiert daraus ein hochqualitatives Mesh.
- **DGtal:** Open-Source Geometrie-Bibliothek zur finalen Centerline-Extraktion. Sie wandelt die Oberfläche des Kabel-Sub-Meshes in eine räumliche 1D-Kurve (Centerline bzw. Scheitelachse mit lokalen Koordinaten) um.

3.2 Pipeline-Struktur: Temporale Konsistenz vs. SAHI

Für die Erkennung extrem dünner Kabel in hochauflösenden 4K-Bildern bietet sich theoretisch *Slicing Aided Hyper Inference* (SAHI) an. SAHI zerschneidet das Bild in kleinere Teilbereiche (Patches), wodurch verhindert wird, dass dünne Kabel beim Downscaling auf die native Modellauflösung von SAM 3 "verschluckt" werden.

Dennoch wurde der primäre Fokus für diese Pipeline bewusst auf den **SAM 3 Video Predictor** gelegt. Der methodische Nachteil von SAHI ist die fehlende temporale Konsistenz: Da SAHI jedes Einzelbild isoliert betrachtet, "flackern" die erzeugten Masken von Frame zu Frame, wenn sich Lichteinfall oder Blickwinkel leicht ändern. 3D Gaussian Splatting erfordert jedoch zwingend formstabile, konsistente Masken über alle Kameraperspektiven hinweg, da sonst fehlerhafte Artefakte in den 3D-Raum projiziert werden. Der SAM 3 Video Predictor löst dieses Problem durch ein Memory-Modul, welches die Kabel-Pixel über die Zeit hinweg verfolgt und dadurch flackerfreie, temporär konsistente

Masken liefert. SAHI dient in dieser Pipeline lediglich als Fallback-Route für spezifische Bildausschnitte, in denen das temporale Tracking abreißen sollte.

4 Docker-Infrastruktur und Orchestrierung

Ein Kernproblem moderner KI-Pipelines sind tiefgreifende Versionskonflikte zwischen den benötigten Nvidia-Grafikkartentreibern (CUDA). Dieses Problem wird durch eine `docker-compose` Architektur vollständig gelöst. Die Pipeline isoliert die Anwendungen in voneinander getrennte Container, während alle erzeugten Daten über **Docker Bind Mounts (Volumes)** zentral und persistent auf dem Host-System (Windows/Linux) gespeichert werden.

Um die Automatisierung zu maximieren, wird die Pipeline über ein zentrales Master-Skript (*One-Click-Orchestration*) gesteuert. Dieses Skript stößt die Container sequenziell an, muss jedoch für manuelle Arbeitsschritte pausieren. Es kommen fünf dedizierte Container zum Einsatz:

- **Container A (Vorverarbeitung):** Beinhaltet PyTorch 2.x und SAM 3. Benötigt das Nvidia Container Toolkit (CUDA 12.8).
- **Container B (SfM):** Ein offizielles NVIDIA-Image für COLMAP (CUDA 11.8 oder 12.1). *Nach Ausführung dieses Containers pausiert das Master-Skript automatisch (Breakpoint). Der Nutzer öffnet CloudCompare auf dem Host-System, pickt die GCPs und speichert die Transformationsmatrix. Anschließend wird das Skript mit "Enter" fortgesetzt.*
- **Container C (Object-specific 3DGS):** Beinhaltet Segment-then-Splat (STS). Empfängt die SAM 3 Masken und die COLMAP-Kameraposen und führt das objektspezifische 3DGS-Training mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$) durch. Benötigt CUDA 12.1 (PyTorch 2.3.1). SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2 als Masken-Frontend.
- **Container D (Meshing):** Beinhaltet SuGaR. Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency) sowie Poisson Surface Reconstruction an. Wegen tiefer Abhängigkeiten zum `diff-gaussian-rasterization` Modul wird hier strikt CUDA 11.8 erzwungen. Das intern geforderte Conda-Environment wird systemweit über eine Anpassung der `ENV_PATH` Variable im Dockerfile nativ zugänglich gemacht.
- **Container E (Post-Processing):** Ein reiner **CPU-Container** (ohne GPU/CUDA-Abhängigkeit). Beinhaltet die C++ Bibliothek DGtal sowie die GDAL-Tools (`ogr2ogr`). Durch die Auslagerung dieser CPU-basierten Nachbearbeitung in einen eigenen Microservice bleibt die Architektur extrem leichtgewichtig, da keine massiven Nvidia-Images geladen werden müssen.

5 Risikomanagement und Datenhandling

Drei kritische Punkte erfordern besondere Aufmerksamkeit:

1. **Speichermanagement:** Drohnenvideos in 4K bei 60 FPS erzeugen massive Datenmengen. Das Konzept sieht vor, dass diese temporär im nativen Linux-Dateisystem liegen und nach der 6 FPS Extraktion gelöscht werden, um I/O-Engpässe unter WSL2 zu vermeiden.
2. **Das Georeferenzierungs-Paradoxon:** Da der modifizierte Mask-Loss den gesamten Hintergrund löscht, verschwinden im 3D-Modell auch die am Boden ausgelegten Passpunkte (GCPs). Eine Post-Training Transformation des Modells durch manuelles Anklicken ist somit nicht anwendbar. Die Lösung ist eine *Pre-Training Georeferenzierung*: Die Open-Source-Software **CloudCompare** bietet hierfür ein ideales GUI. Man lädt die unmaskierte, spärliche SfM-Punktwolke (*Sparse Point Cloud*) aus COLMAP hinein, ordnet die GCPs manuell zu (*Point Picking*) und tippt die bekannten GNSS-Koordinaten ein. CloudCompare berechnet daraus eine präzise Transformationsmatrix. SuGaR berechnet danach parallel das rohe, lokale 3D-Mesh. Anstatt jedoch das rechenintensive Millionen-Polygon-Mesh in globale Koordinaten zu überführen, wird zunächst DGtal auf dem lokalen Mesh ausgeführt. Die resultierende lokale Centerline (bestehend aus wenigen hundert Punkten) wird anschließend mit der Transformationsmatrix multipliziert und springt ressourcenschonend und fehlerfrei auf die globalen UTM-Koordinaten.
3. **[NEUE ERKENNTNIS: Datenformate & GIS-Export]**
 ArcGIS Pro unterstützt .ply-Geometrien nicht nativ als Feature-Layer. Zudem verursacht das .obj-Format bei großen UTM-Koordinaten signifikante Präzisionsverluste (*Vertex Jittering*), da es standardmäßig als *Single Precision* verarbeitet wird. Die Pipeline splittet den Datenexport daher gezielt in zwei Endprodukte auf:
 - **Das analytische Endprodukt (Centerline):** Da DGtal (als diskrete Geometrie-Bibliothek) im lokalen Raum um den Ursprung (0,0,0) am präzisesten und speichereffizientesten arbeitet, wird die Centerline zuerst auf dem lokalen .ply-Mesh berechnet. Erst diese 1D-Kurve wird mit der CloudCompare-Matrix georeferenziert. Die Konvertierung des Ergebnisses in standardisierte GIS-Formate (wie GeoJSON oder Shapefile) erfolgt Open-Source über den **GDAL/ogr2ogr Post-Processing Container**, wodurch proprietäre Software wie FME überflüssig wird.
 - **Das visuelle Endprodukt (BIM-Mesh):** Falls TenneT das Kabel zusätzlich als 3D-Objekt visualisieren möchte, wird über das nativ in ArcGIS Pro integrierte Geoprocessing-Tool Import 3D Files ein lokales .obj eingelesen und direkt in der Software an die richtige Koordinate gerückt (mittels *Anchor Point / Global Shift*), was den Präzisionsverlust des .obj-Formats elegant umgeht.

6 Zeit- und Scope-Planung

Projektarbeit (PA) – Fokus Machbarkeit (4 Wochen):

- Konfiguration der Docker-Architektur (Container A, B, C & D).
- Master-Skript Orchestrierung inkl. manuellem Breakpoint.

- Evaluierung von SAM 3: Hauptweg vs. Fallback (SAHI).
- Nachweis der Funktionalität der Mask-Loss Modifikation.

Bachelorarbeit (BA) – Fokus Skalierung & Metrik (10 Wochen):

- **Feldarbeit:** Drohnenflug sowie parallele GNSS-Referenzmessung der ausgelegten GCPs und der Kabel-Scheitelachse (*Centerline*).
- Georeferenzierung via *Point Picking* in CloudCompare (Nutzung der GNSS/GCPs).
- Centerline-Extraktion via DGtal auf dem lokalen Mesh.
- **Wissenschaftliche Evaluation (Metriken):** Berechnung des Root Mean Square Error (**RMSE**) und der maximalen **Hausdorff-Distanz** der extrahierten 3DGS-Centerline gegenüber einer interpolierten **Basis-Spline-Kurve (B-Spline)** aus den diskreten GNSS-Referenzmessungen, um die Einhaltung der ± 10 cm Toleranz nachzuweisen.
- Wirtschaftlichkeitsanalyse für TenneT.

7 Anhang: Single Source of Truth (SSOT) – Pipeline Installation

Dieses Dokument dient als zentrale Wahrheit (SSOT) für die Installation und Orchestrierung der gesamten 4-Container-Pipeline. Es fasst alle Hard- und Software-Abhängigkeiten zusammen, um Reproduzierbarkeit zu gewährleisten.

Hardware-Grundvoraussetzungen

- **GPU:** CUDA-kompatible Nvidia GPU (Compute Capability 7.0+).
- **VRAM:** 24 GB (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).
- **OS:** Windows 10/11 mit WSL2 oder natives Ubuntu Linux 22.04.

Container A: SAM 3 (Segmentierung)

Zuständig für die 2D-Maskierung des Drohnenvideos. Dieser Container nutzt modernste Meta-Abhängigkeiten.

Requirements (laut offizieller Vorgabe):

- Python 3.12 or higher
- PyTorch 2.7 or higher (im Setup wird `torch==2.10.0` mit `cu128` installiert)
- CUDA-compatible GPU with CUDA 12.6 or higher

Installationsablauf (Conda & Pip):

```
conda create -n sam3 python=3.12
conda deactivate
conda activate sam3
pip install torch==2.10.0 torchvision --index-url https://download.pytorch.org/whl/cu128
git clone https://github.com/facebookresearch/sam3.git
cd sam3
```

```

pip install -e .
# For running example notebooks
pip install -e "[notebooks]"
# For development
pip install -e "[train,dev]"
# Optional dependencies for faster inference
pip install einops ninja && \
    pip install flash-attn-3 --no-deps --index-url https://download.pytorch.org/whl/cu
pip install git+https://github.com/ronghanghu/cc_torch.git

# Model Checkpoints (SAM 3.1) in den Container/Volume herunterladen
mkdir -p checkpoints
cd checkpoints
# Download der aktuellsten SAM 3.1 Checkpoints (Object Multiplex) via wget
wget -q https://huggingface.co/facebook/sam3-hiera-large/resolve/main/sam3.1_hiera_la
cd ..

```

Container B: COLMAP (Structure from Motion)

Generiert die Kameraposen und die Sparse Point Cloud aus den maskierten Drohnenbildern. Aufgrund der massiven Architekturänderungen (Einführung von GLOMAP) in Release 4.0.0, wird explizit zur stabileren Patch-Version **4.0.4** geraten, welche kritische Bugs (Speicherlecks, Race-Conditions in der Datenbank) behebt.

Quellen & Ausführung:

- **Releases:** <https://github.com/colmap/colmap/releases>
- **Docker-Image:** `docker pull colmap/colmap:4.0.4-cuda`

Container C: Segment-then-Splat (Object-specific 3DGS)

Führt das objektspezifische 3DGS-Training durch. STS weist jedem initialen COLMAP-Gaussian eine Object-ID zu (basierend auf den SAM 3 Masken) und trainiert mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$). Das intern vorgesehene AutoSeg-SAM2 wird durch die externen SAM 3 Masken aus Container A ersetzt.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:12.1.0-devel-ubuntu22.04`
- Python 3.10, PyTorch 2.3.1 mit CUDA 12.1
- 24 GB VRAM

Installationsablauf:

```

git clone https://github.com/luyr/Segment-then-Splat --recursive
cd Segment-then-Splat

```

```

conda create -n segment_then_splat python=3.10
conda activate segment_then_splat
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 \

```

```
--index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt --no-build-isolation
```

Ausführung (Preprocessing + Training):

```
# 1. Masken-Preprocessing (SAM 3 Masken aus Container A)
python ./helpers/preprocess_mask.py \
    --mask_root /data/masks \
    --out_root /data/scene/ \
    --image_path /data/scene/images

# 2. Object-specific Initialization (benötigt COLMAP-Output)
python ./helpers/object_specific_initialization.py \
    --scene_root /data/scene/

# 3. Training mit Object-specific Loss
python train.py -s /data/scene/ -m /data/output \
    --eval --iterations 40000 \
    --num_sample_objects 3 \
    --densify_until_iter 20000 \
    --partial_mask_iou 0.3
```

Wichtiger Hinweis: Das Repository ist mit dem Vermerk “(*To be verified*)” beim Object-Tracking markiert. Die Stabilität der Preprocessing-Skripte muss im Rahmen der Projektarbeit validiert werden.

Container D: SuGaR (Geometrische Regularisierung & Meshing)

Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. Dies ist der sensitivste Container bezüglich CUDA-Versionen.

Exkurs: Conda innerhalb von Docker

Obwohl Docker bereits das Betriebssystem isoliert und Conda somit theoretisch redundant wäre, wird Miniconda in diesem Container explizit genutzt. Der Grund: 3DGS-Repositories (wie SuGaR) setzen stark auf vorkompilierte Conda-Binaries (z.B. `pytorch-cuda`) und exakte Paket-Versionen. Durch die strikte Nutzung von Conda im Docker-Container werden fatale C++ Kompilierungsfehler beim Rasterizer-Modul präventiv vermieden.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:11.8.0-devel-ubuntu22.04` (Dieses Image bringt das kompatible CUDA 11.8 SDK direkt mit).
- **C++ Compiler:** Installation via `apt-get install build-essential` (installiert GCC/G++, was unter Linux Visual Studio ersetzt und garantiert mit CUDA 11.8 kompatibel ist).
- 24 GB VRAM (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).

Installationsablauf (Dockerfile / Skript):

```
# 1. Base Image mit CUDA 11.8
FROM nvidia/cuda:11.8.0-devel-ubuntu22.04

# 2. C++ Compiler (GCC) installieren
RUN apt-get update && apt-get install -y build-essential git wget

# 3. Python & Conda installieren und Environment aufbauen
conda create --name sugar -y python=3.9
conda activate sugar
conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 \
    pytorch-cuda=11.8 -c pytorch -c nvidia
conda install -c fvcore -c iopath -c conda-forge fvcore iopath
conda install pytorch3d==0.7.4 -c pytorch3d
conda install -c plotly plotly
conda install -c conda-forge rich plyfile==0.8.1 jupyterlab nodejs ipywidgets
pip install open3d PyMCubes

# 4. Rasterizer & KNN kompilieren (nutzt nun automatisch GCC)
cd gaussian_splatting/submodules/diff-gaussian-rasterization/
pip install -e .
cd ../simple-knn/
pip install -e .
cd ../../../../

# 5. Optional: Nvdiffrast (für extrem schnellen Mesh Export)
git clone https://github.com/NVlabs/nvdiffrast
cd nvdiffrast && pip install . && cd ../
```

Wichtige Best-Practices & Parameter-Tuning (SuGaR):

Für qualitativ hochwertige Rekonstruktionen (insb. dünne Kabelstrukturen) sind folgende Parameter entscheidend (siehe SuGaR Tips):

- **Regularisierung:** Nutze zwingend `-r "dn_consistency"` für die beste Mesh-Qualität.
- **Löcher im Mesh:** Wenn die Mesh-Bereinigung zu aggressiv ist, setze `vertices_density_quantile = 0.` in `sugar_extractors/coarse_mesh.py`.
- **Ellipsoide Artefakte (Bumps):** Reduziere die Poisson-Tiefe `poisson_depth` (z.B. von 10 auf 7 oder 8 in `coarse_mesh.py`), falls einzelne Gaussians sichtbar werden.
- **Große Szenen:** Bei weiten Distanzen sollten Custom Bounding Boxes (`-bboxmin` und `-bboxmax`) genutzt werden, um die Extraktion zu fokussieren.

Container E: DGtal & GDAL (Post-Processing)

Führt die Extraktion der Centerline auf dem isolierten Kabel-Sub-Mesh durch und überführt diese in georeferenzierte GIS-Formate (GeoJSON). Da `DGtal` eine komplexe C++ Bibliothek mit zahlreichen Abhängigkeiten (Eigen, CGAL, ITK) ist, wird direkt auf dem offiziellen `dgtal:latest` Docker-Image aufgebaut.

Requirements & Repositories:

- **DGtal Release (v2.1):** <https://github.com/DGtal-team/DGtal/releases/tag/v2.1>
- **GDAL:** <https://github.com/OSGeo/gdal>
- **OGR2OGR Wrapper:** <https://github.com/wavded/ogr2ogr>

Docker Build & Ausführung:

```
# Offizielles DGtal Image bauen (im DGtal Repo Ordner)
```

```
docker build -t dgtal:latest .
```

```
# Nachinstallation von GDAL (ogr2ogr) über ein abgeleitetes Dockerfile
```

```
FROM dgtal:latest
```

```
USER root
```

```
RUN apt-get update && apt-get install -y gdal-bin libgdal-dev && \
    rm -rf /var/lib/apt/lists/*
```

```
USER digital
```

```
# Interaktive Ausführung mit Bind-Mount
```

```
docker run -it -v /host/path/data:/data --user=digital dgtal-gdal:latest bash
```

```
cd /home/digital/git/DGtal/build/examples
```

```
# Nach DGtal Lauf:
```

```
# ogr2ogr -f "GeoJSON" /data/centerline.geojson /data/centerline.csv -a_srs EPSG:25832
```

Exkurs: Kritische Evaluation des MMask-Loss Hacks in Kombination mit SAM 3

Obwohl die isolierte Rekonstruktion der Kabeltrasse durch Segmentierungsmasken (z.B. aus SAM 3) äußerst ressourceneffizient erscheint, offenbart eine präzise Code-Analyse des SuGaR/3DGS-Repositories, dass es sich bei dem oft zitierten MMask-Loss Hack nicht um eine echte Maskierung der mathematischen Loss-Funktion handelt. Vielmehr ist es ein fehleranfälliger Workaround bei der Datenvorbereitung. Die ingenieurstechnische Zuverlässigkeit dieses Vorgehens muss im Kontext der geforderten geometrischen Toleranzen (± 10 cm) kritisch bewertet werden.

1. Der Mechanismus: Manipulation der Ground-Truth Bilder

Eine Überprüfung der Pipeline-Skripte (insbesondere `scene/cameras.py` und `train.py`) zeigt, dass die Loss-Funktionen (L1 und SSIM)¹ die Maske während der Backpropagation nicht berücksichtigen. Stattdessen werden die Originalbilder (Ground-Truth) beim Initialisieren im Speicher direkt mit der binären SAM 3 Maske multipliziert (`original_image *= gt_alpha_mask`). Dies überschreibt alle Hintergrundpixel unwiderruflich mit dem RGB-Wert `[0, 0, 0]` (pechschwarz). Die Loss-Funktion berechnet anschließend den Fehler

¹**L1-Loss:** Beschreibt die mittlere absolute Abweichung (Mean Absolute Error) der einzelnen Pixelwerte zwischen dem gerenderten Bild und der Ground-Truth. **SSIM (Structural Similarity Index Measure):** Misst die wahrgenommene strukturelle Ähnlichkeit zweier Bilder anhand von Leuchtdichte, Kontrast und lokalen Mustern, was der menschlichen visuellen Wahrnehmung näher kommt als reine Pixelabweichungen.

über das *gesamte* Bild, inklusive des künstlich eingefügten schwarzen Raumes. Um den Loss zu minimieren, ist das 3DGS-Modell somit gezwungen, dieses schwarze Vakuum aktiv zu lernen, anstatt es schlichtweg zu ignorieren.

2. Das SSIM-Fenster Problem (Artefakt-Bildung an Rändern)

Die 3DGS-Pipeline nutzt für die strukturelle Bildauswertung den SSIM-Loss, der mit einem 11×11 Pixel großen Convolution-Fenster arbeitet. An den harten Kanten der SAM 3 Maske, wo die reale Textur des Kabels direkt auf den künstlichen schwarzen Hintergrund trifft, erfasst dieses Fenster unweigerlich beide Bereiche. Dies zwingt das neuronale Netz dazu, einen unnatürlich scharfen räumlichen Gradienten in den 3D-Raum zu projizieren. In der Praxis führt dies dazu, dass das Modell dunkle, halbdurchsichtige Splats ("Black Shells oder FFloater") eng um das Kabel herum aufbaut, um diesen extremen Helligkeitsübergang zum schwarzen Hintergrund zu simulieren. Diese Artefakte verfälschen die tatsächliche Kabelgeometrie maßgeblich und machen eine spätere fehlerfreie Centerline-Extraktion nahezu unmöglich.

3. Die Imperfektion von SAM 3 (Pixel-Bluten & Geometrie-Stauchung)

Obwohl SAM 3 durch das Memory-Modul eine exzellente temporale Konsistenz aufweist, sind die generierten Masken an den Rändern selten auf den Subpixel genau. Dies hat unweigerlich direkte geometrische Konsequenzen für das finale Mesh:

- **Oversampling (Maske minimal zu groß):** Umfasst die Maske auch nur wenige Pixel des umliegenden Baugrabens, werden diese Erdstrukturen in das schwarze Vakuum übernommen und als schwebende "Erdrumpfen" direkt am Kabel mit rekonstruiert.
- **Undersampling (Maske minimal zu klein):** Wenn die Maske Teile des echten Kabels abschneidet, entsteht in der Ground-Truth ein künstlicher schwarzer Bereich, wo in der Realität das Kabel verläuft. Dies zwingt das 3DGS-Modell dazu, die Splats des echten Kabels an dieser Stelle in der Tiefe zu stauchen oder vollständig zu unterdrücken, da eine korrekte 3D-Rekonstruktion an dieser Stelle einen hohen L1-Loss gegenüber dem fehlerhaften schwarzen 2D-Zielbild erzeugen würde.

Da die DGtal Centerline-Extraktion extrem sensitiv auf die exakte lokale Oberfläche des Meshes reagiert, pflanzen sich diese durch die 2D-Maske verursachten 3D-Geometriefehler (Stauchungen/Floater) direkt als Abweichung in die finalen UTM-Koordinaten fort.

4. Das Random-Background Dilemma

Standardmäßig trainiert 3DGS mit zufällig gefärbten Hintergründen (`random_background = True`), was als essenzielle Regularisierung dient, um die Bildung von schwebenden Artefakten (Floater) im leeren Raum zu verhindern. In Kombination mit den schwarz maskierten Ground-Truth-Bildern führt diese Regularisierung jedoch zu einem katastrophalen Architekturversagen: Das Modell ist gezwungen, massive Wände aus tiefschwarzen Splats in den leeren Raum zu bauen, um den zufällig gefärbten Rendering-Hintergrund vor der

Kamera zu verdecken, nur um das künstlich schwarze Zielbild zu erreichen. Der MMask-Loss Hackßwingt den Anwender folglich dazu, den Hintergrund beim Rendern zwingend und permanent auf Schwarz zu fixieren. Dadurch verliert das Modell seine natürliche Regularisierung gegen Artefakte.

5. Limitationen nachgelagerter Fehlerkorrekturen (Glättung & Splat-Filtering)

Es könnte argumentiert werden, dass die beschriebenen Artefakte durch nachgelagerte Post-Processing-Schritte behoben werden können. Zwei gängige Ansätze erweisen sich jedoch im Kontext strenger Toleranzen (± 10 cm) als unzureichend:

- **Glättung der Centerline:** Eine mathematische Glättung der extrahierten 1D-Kurve (z.B. mittels B-Splines) korrigiert lediglich statistisches Rauschen (Varianz). Die durch den unsauberen Mask-Loss induzierten Artefakte ("Black Shells" und Stauchungen) verursachen jedoch einen *systematischen Fehler* (Bias) der Kabeloberfläche. Eine geglättete Kurve auf Basis einer systematisch verschobenen Oberfläche führt unweigerlich zu einer inkorrekten räumlichen Lage.
- **Post-Processing Splat-Filtering:** Eine naheliegende Idee ist das algorithmische Löschen aller schwarzen Gaussians (z.B. RGB-Werte $\leq [5, 5, 5]$) aus der .ply-Punktwolke vor dem Meshing. Dieser Workaround birgt jedoch das hohe Risiko, reale beschattete oder dunkle Kabelabschnitte (False Positives) ebenfalls zu löschen, was das Meshing durch SuGaR aufgrund fehlender Punkte massiv stört. Zudem heilt das nachträgliche Löschen dunkler Splats nicht die geometrische Stauchung der *verbleibenden* Kabel-Splats, die während des Trainings aufgrund der manipulierten Ground-Truth bereits irreversibel entstanden ist.

Brutales Fazit & Implementierungs-Roadmap: Die bloße Kombination aus SAM 3 Masken und der Manipulation der Ground-Truth-Bilder (der aktuelle "Mask-Loss Hack" in SuGaR) ist aus ingenieurstechnischer Sicht nicht ausreichend, um strenge Toleranzen im Millimeter/Zentimeter-Bereich sicherzustellen. Der Ansatz ist bestenfalls ein fehleranfälliger Workaround, der geometrische Stauchungen und SSIM-Randartefakte provoziert.

Die Lösung ist eine konsequente Zerteilung der 3D-Rekonstruktion: *Segment-then-Splat* (Lu et al., NeurIPS 2025)² übernimmt als **Container C** das objektspezifische 3DGS-Training mit einem nativen Object-specific Loss ($\mathcal{L}_{obj}$), der die Ground-Truth-Bilder nicht manipuliert. STS teilt die initialen COLMAP-Gaussians *vor* dem Training anhand der SAM 3 Masken in objektspezifische Sets auf und eliminiert damit sämtliche in den vorherigen Abschnitten beschriebenen Artefakte architektonisch. Allerdings basiert STS auf Vanilla 3DGS und liefert keine geometrische Regularisierung.

Daher wird der vortrainierte STS-Checkpoint anschließend an **SuGaR (Container D)** übergeben. SuGaR wurde exakt für diesen Workflow konzipiert: Es lädt einen bestehenden 3DGS-Checkpoint und wendet zusätzliche Regularisierungs-Losses (DN-Consistency, Normal Alignment) an, um die Gaussians flach an die reale Oberfläche zu zwingen. Abschließend extrahiert SuGaR via Poisson Surface Reconstruction ein hochqualitatives

²Y. Lu et al., "Segment then Splat: Unified 3D Open-Vocabulary Segmentation via Gaussian Splatting", NeurIPS 2025. <https://github.com/luyr/Segment-then-Splat>

Mesh. Da jeder Gaussian durch STS bereits eine Object-ID trägt, kann das Kabel-Sub-Mesh nach dem Meshing anhand dieser IDs isoliert und an DGtal (Container E) zur Centerline-Extraktion übergeben werden.

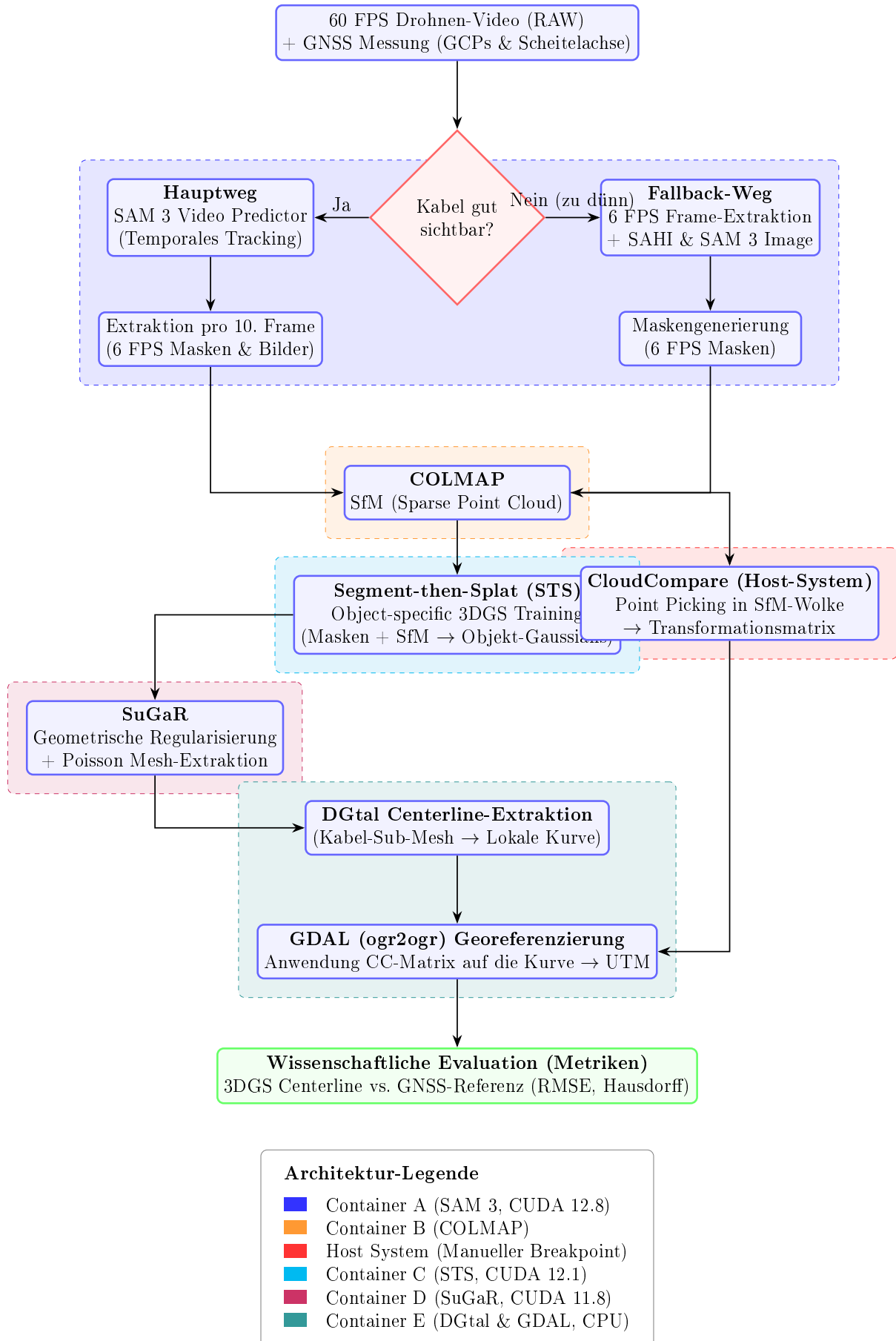


Abbildung 1: Microservice-Architektur (v4): 5-Container-Pipeline mit Segment-then-Splat für objektspezifisches Training und SuGaR für geometrisches Meshing.